

COMP 3700
Software Modeling & Design

Chapter4: Use Case
Dr. Xuechao Li



AUBURN

UNIVERSITY

SAMUEL GINN
COLLEGE OF ENGINEERING

Outline

- What are use cases?
- How to do it?

What are use cases?

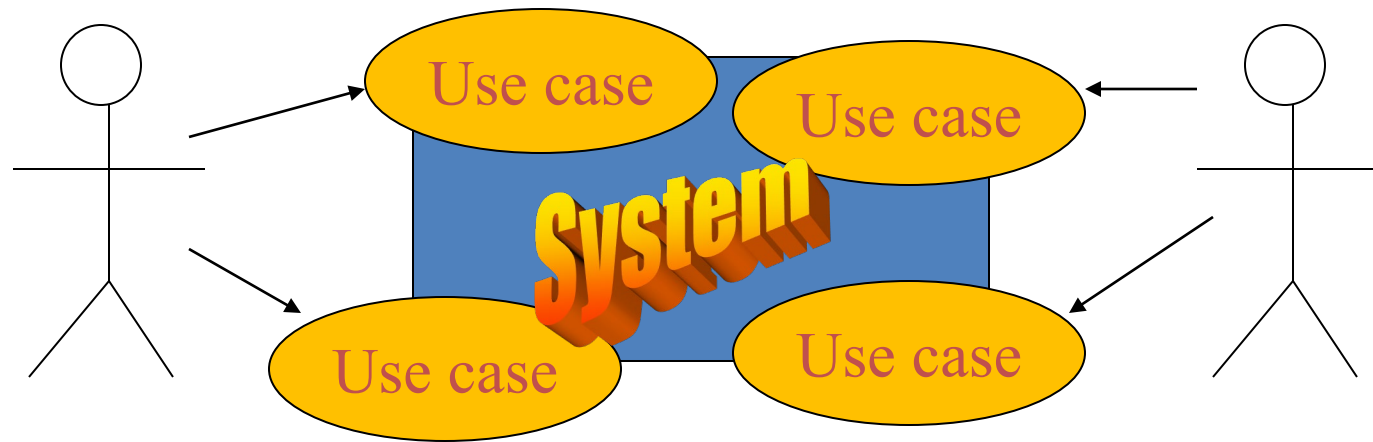
- A use case is an envelope of scenarios
 - Use case name is goal statement
 - Order product from catalogue
 - Scenarios are the stories of attempts at the goal
 - “sunny day”-everything works out
 - Agent is not authorized
 - Customer has insufficient credit
 - Use case is goal plus scenarios
 - Each use case has many scenarios
 - Each scenarios is described in terms of steps
 - The scenario steps form a directed graph, and every path (scenario) either succeeds in realising the goal, or fails in some way
 - Use cases represent (textually) the graph as a whole rather than each scenario separately

What are use cases?

- Use case model is made up of
 - Preamble
 - Introducing any general considerations, style, assumptions that will aid the reader
 - Actor list
 - Role names and brief descriptions
 - Use case list
 - Names and unique ids
 - Set of use cases
 - Using the template
 - Issues
 - For the use case model (not for the project)

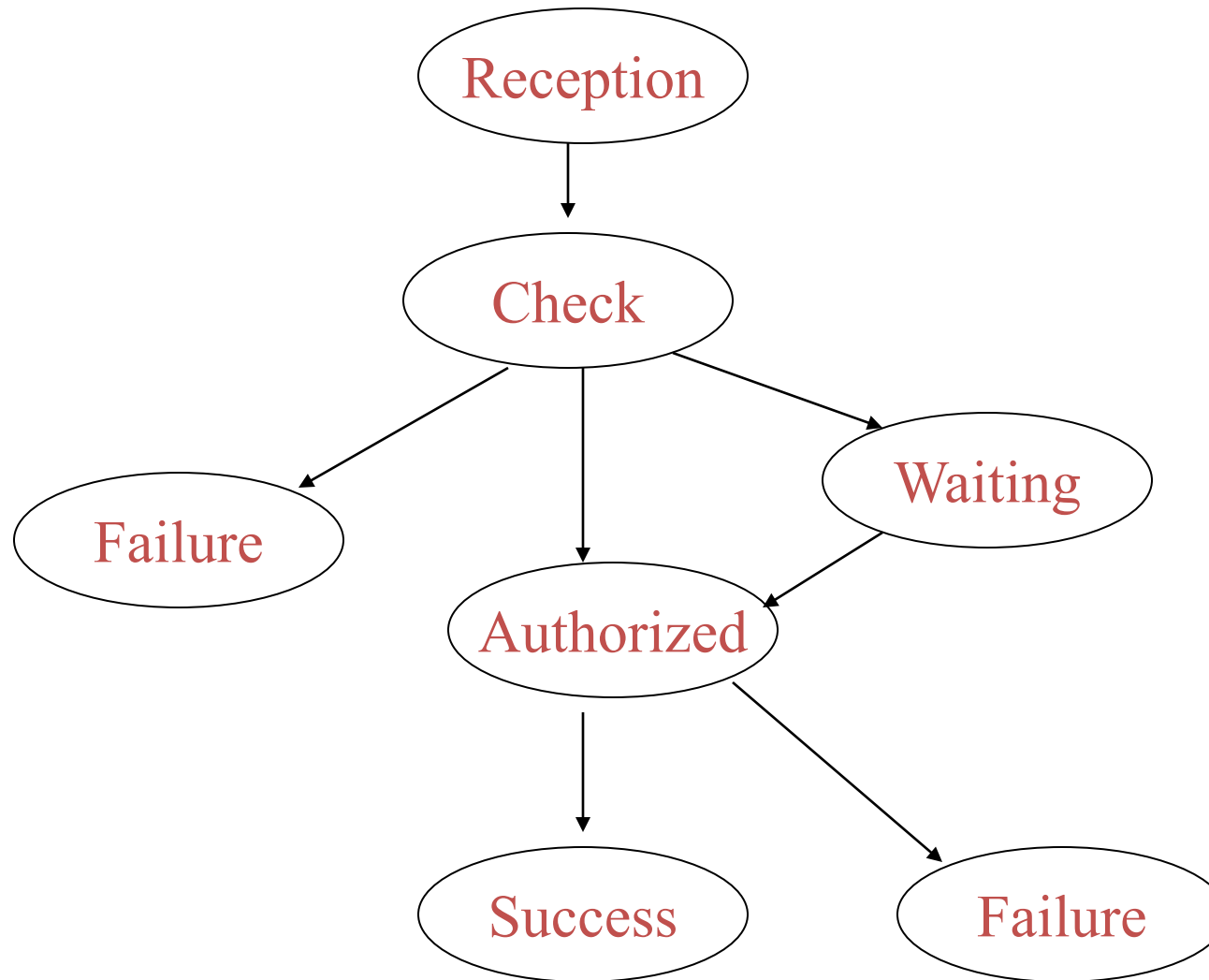
What are use cases?

- Each use case defines an interaction between an actor and the system



What are use cases?

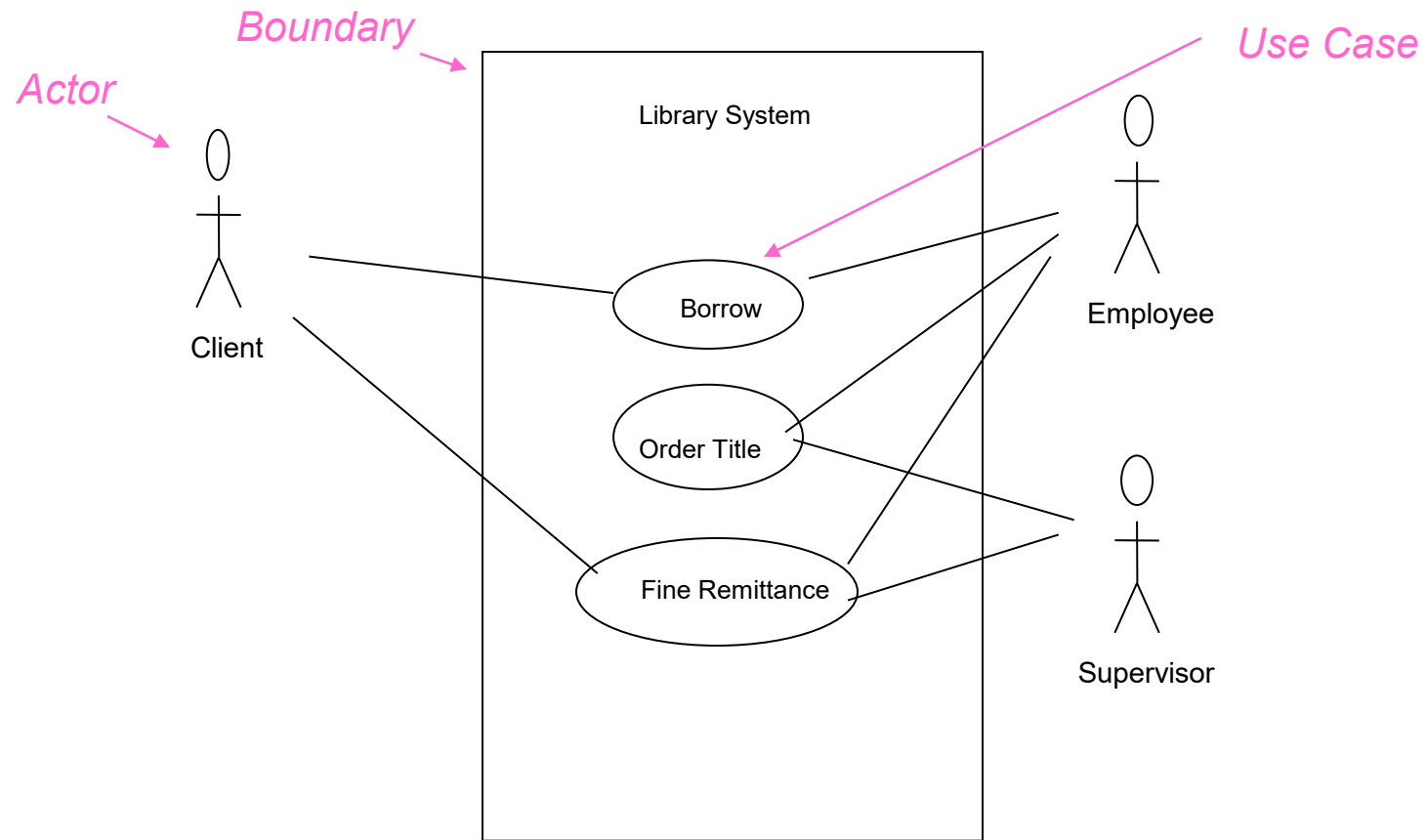
- Scenarios of a use case (stories)



Use-Case Diagrams

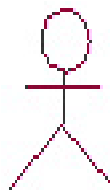
- A use-case diagram is a set of use cases
- A use case is a model of the interaction between
 - External users of a software product (actors) and
 - The software product itself
 - More precisely, an actor is a user playing a specific role
- describing a set of user **scenarios**
- capturing user requirements
- **contract** between end user and software developers

Use-Case Diagrams

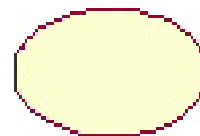


Use-Case Diagrams

- **Actors:** A role that a user plays with respect to the system, including human users and other systems. e.g., inanimate physical objects (e.g. robot); an external system that needs some information from the current system.
- **Use case:** A set of scenarios that describing an interaction between a user and a system, including alternatives.
- **System boundary:** rectangle diagram representing the boundary between the actors and the system.



Actor



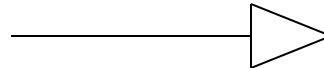
Use Case

Use-Case Diagrams

- Association: communication between an actor and a use case; Represented by a solid line.



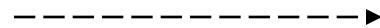
- Generalization: relationship between one general use case and a special use case (used for defining special alternatives); Represented by a line with a triangular arrow head toward the parent use case.



Use-Case Diagrams

- **Include**: a dotted line labeled <<include>> beginning at base use case and ending with an arrows pointing to the include use case. The include relationship occurs when a chunk of behavior is similar across more than one use case. Use “include” in stead of copying the description of that behavior.

<<include>>

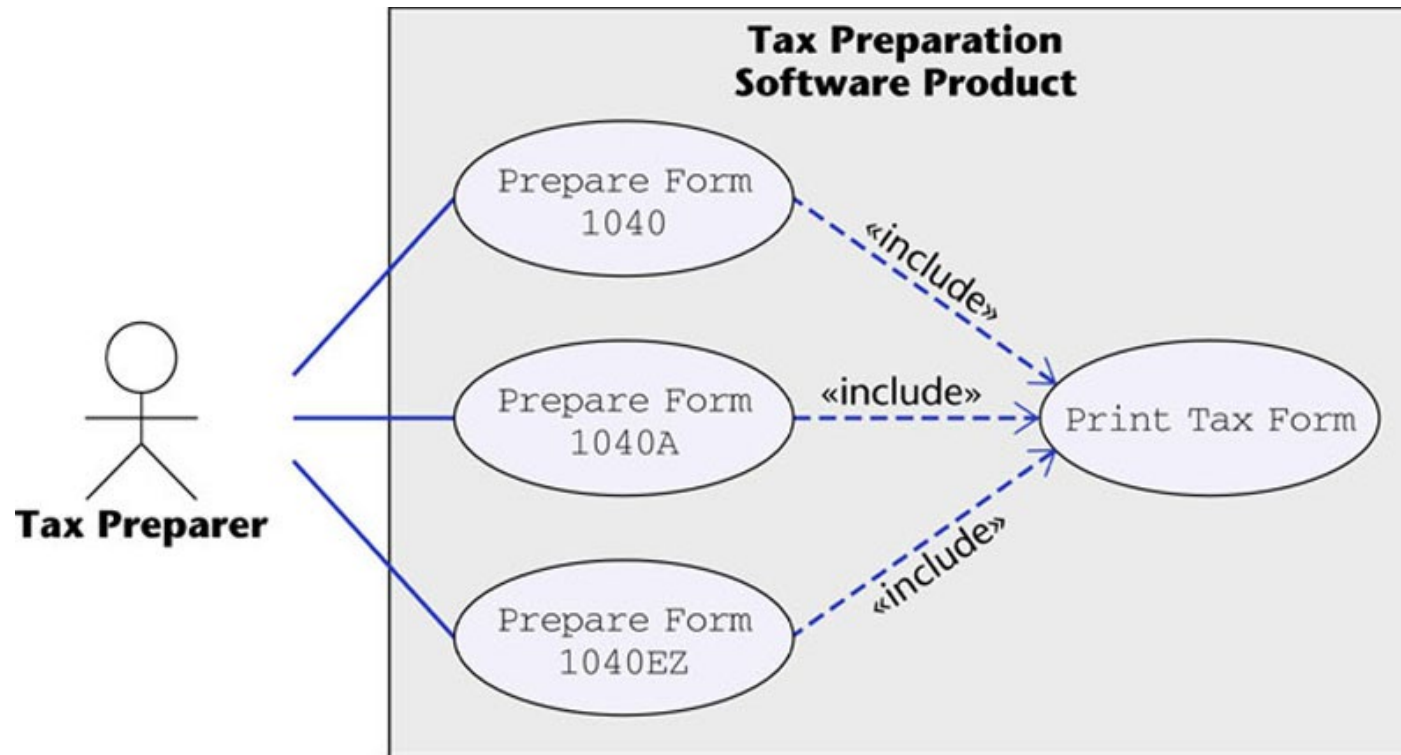


- **Extend**: a dotted line labeled <<extend>> with an arrow toward the base case. The extending use case may add behavior to the base use case. The base class declares “extension points”.

<<extend>>



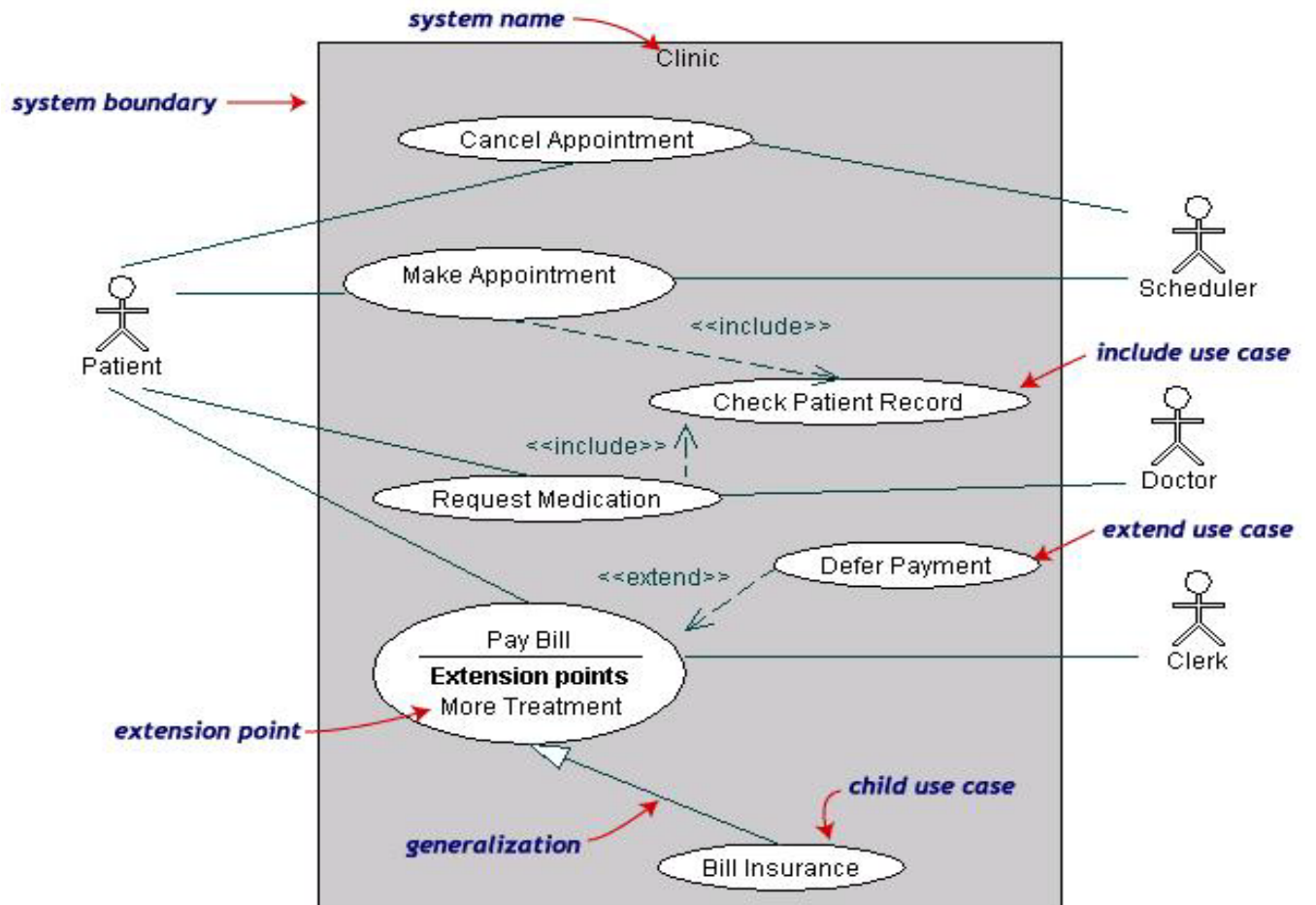
Use-Case Diagrams



The McGraw-Hill Companies, 2005

Use-Case Diagrams

- Both **Make Appointment** and **Request Medication** include **Check Patient Record** as a subtask (include)
- The **extension point** is written inside the base case **Pay bill**; the extending class **Defer payment** adds the behavior of this extension point. (extend)
- **Pay Bill** is a parent use case and **Bill Insurance** is the child use case. (generalization)



How to do it – four steps

- Identify primary actors and goals
- Write a scenario (path) in which the goal is delivered
- Identify failure conditions for scenario steps
- Follow each alternative (exception) until it ends or rejoins

Step ¼ Primary actors and goals

- Identify primary actors and goals
 - Actors are roles, not people or things
 - A single person may be many actors
 - Any interaction across a system boundary implies an actor
- Ask: what people and other systems will drive our system?
 - Result: a list of actor names and brief descriptions
- Ask: what does each actor want our system to do? (The goal of each actor when using our system)
 - Result: a list of use case names, top-level coverage of functional requirements

Capturing Actors

- An actor is a role
 - Financial Planning Manager (insurance salesman)
 - Bank teller
- Simple to capture
 - Actor name
 - Brief description
- Listing the actors helps with completeness
 - Covering all the uses of the system
- Primary actors are those which initiate use cases

Advice & guidance

- Name the use cases
- Remember to think about actors
- Express goals from point of view of actor
- Use the “1 person, 1 place, 1 time” rule of thumb to judge the appropriate granularity at which to identify business goals / use cases
- The system doesn’t know which actor is the user (do not include name of actor in use case name)
- Don’t worry too much about secondary actors

Secondary actors

- Secondary actors are used by the system under design
- Will appear on the system context diagram
- May be mentioned in the Non-functional requirements
- Usually secondary actors are system roles

Step 2/4 Simple goal delivered

- For each use case, write a scenario in which the goal is delivered
- The main success scenario, the happy day case
 - Easies to read and understand
 - Everything else is a complication on this
- Capture the actor intent and system responsibility through to goal delivery
 - Say what information passes between them
 - Number each line
 - User, system, user, system, ...
- Result: coverage of function of each use case

Example

- Place an order (UC1)
 - Clerk identifies customer
 - System lists customer account details
 - Clerk creates order
 - System verifies and submits order
 - Use case end successfully

Preconditions

- A precondition is an assumption which must be true before a use case is used
- Preconditions are not checked within a use case
- Use preconditions to express dependencies between use cases
 - *What must have happened before the start of the main scenario? For example: clerk is logged on.*

Steps refer to lower-level goals

- Main scenario: clerk identifies customer (UC7)
- UC77:
 - System prompts for customer name
 - Clerk enters customer name
 - System lists matching customer name and address
 - Clerk selects customer
 - End successfully
- Alternatives (exceptions)
 - Customer name is not found
 - System reports
 - Continue at step 1
 - Use case ends in failure (account expired)

Step $\frac{3}{4}$ Failure conditions

- Identify potential failure conditions for scenario steps
- Ask: what can go wrong?
- Result: list of conditions which will be used to develop alternative scenarios

Failure scenarios help with completeness

- Particular questions we might ask
 - What if their credit is too low?
 - What if they run over the extended credit limit?
 - What if we cannot deliver the quantity?
 - What if the order is placed too late?
- Such questions are commonly overlooked

Step 4/4 Alternative scenarios

- Each failure condition introduces an alternative scenario
- Decide at which step the new scenario diverges
- Follow the alternative until it ends or rejoins
- Recoverable alternatives rejoin the main course
- Non-recoverable alternatives end use case in failure
- Result: use cases expressed as set of scenarios

Summary

- Functional requirements have got to be captured somehow
- It encourages asking the right questions at the right time
- It's systematic
 - To check coverage or evenness of details
- The results are quickly navigated
 - Top-level is absorbed very quickly
 - Explore details where needed
 - Uniformity of format simplifies exploration of details